

04. Onboarding and Setup

Dynamic Workload Allocation Between Space and Ground

Team [pas encore]

Version 1.0 - September 2026

Table des matières

1	Accounts and Access (Day 1 Checklist)	2
2	Environment	2
2.1	Installs (Updated)	2
3	Repository Layout and Starter Template	2
3.1	Experiment Configuration (<code>config/experiment.yaml</code>)	3
3.2	Shared State Schemas (<code>src/schemas.py</code>)	3
3.3	Strategy Interface (<code>src/strategies/base.py</code>)	4
4	Strategy 5 and Visualization Extensions	5
4.1	Extending the Starter Code for Strategy 5	5
4.2	Interactive Dashboard	5
4.3	Optional : Using Skyfield in Phase 3	5
5	Minimal Working Example (Our First Task)	6

1 Accounts and Access (Day 1 Checklist)

- ☐ GitHub/GitLab account added to the project repository.
- ☐ Python 3.11+ installed locally.
- ☐ Code editor (VSCode or PyCharm) configured with a virtual environment.

2 Environment

All simulation runs can be executed locally on a standard CPU. For heavier batch runs (Phase 3), Google Colab CPU instances or any standard laptop is sufficient.

2.1 Installs (Updated)

Create a `requirements.txt` in your project root, including `skyfield` for the optional orbital calculations :

```
1 simpy>=4.1.0
2 pydantic>=2.7
3 pandas>=2.0
4 numpy>=1.26
5 pyyaml>=6.0
6 matplotlib>=3.8
7 plotly>=5.0           # Interactive Gantt charts and Pareto dashboards
8 torch>=2.0            # Backend for reinforcement learning
9 stable-baselines3>=2.0 # DQN/PPPO implementations
10 skyfield>=1.48        # Orbital mechanics (TLE data)
```

Listing 1 – requirements.txt (updated)

Install via terminal :

```
1 pip install -r requirements.txt
```

3 Repository Layout and Starter Template

```
orbital-scheduler/
|-- config/
|   |-- experiment.yaml      # Strategies, profiles, seeds, number of runs
|   '-- topology.yaml       # Nodes, links, capacities (from doc 02, Section 2)
|-- data/
|   |-- psplib_j30.sm        # PSPLIB instance
|   |-- google_sample.csv    # Pre-processed arrival timestamps
|   '-- tle.txt              # TLE data for Skyfield
|-- docs/                    # Documents 01-04 + README
|-- src/
|   |-- schemas.py           # Pydantic state models (shared, do not fork)
|   |-- env/
|   |   |-- node.py          # Satellite / GroundStation / Cloud classes
|   |   |-- link.py          # Link with availability windows
|   |   |-- task_generator.py # Synthetic task generator
|   |   '-- real_benchmark_loader.py # PSPLIB + Google + Skyfield
|   |-- strategies/
|   |   |-- base.py          # Abstract Strategy class
|   |   |-- s1_always_ground.py # Team 1
|   |   |-- s2_always_orbital.py # Team 1
|   |   |-- s3_greedy.py      # Team 1
|   |   |-- s4_dynamic.py     # Team 1 (contains re-scheduler)
|   |   '-- s5_rl_agent.py    # RL policy wrapper (Team 2)
|   |-- visualization/
```

```

| | | -- dashboard.py          # Interactive Plotly dashboard
| | | -- gantt_chart.py        # Dynamic Gantt chart generator
| | | -- pareto_plot.py        # Pareto frontier visualizer
| | -- orchestrator.py         # Main SimPy process that calls strategies
| | -- metrics/
| | | -- calculator.py          # Computes M_T, M_L, M_R, M_E, M_S
|-- notebooks/                 # Jupyter/Colab entry points
|-- telemetry/                 # JSONL outputs (gitignored)

```

3.1 Experiment Configuration (config/experiment.yaml)

```

1 benchmark_mode: "synthetic" # "synthetic" or "merged"
2 simulation_runtime: 3600    # 1 hour simulated time
3 num_runs_per_cell: 5       # For statistical averaging
4
5 strategies_to_run: [1, 2, 3, 4, 5]
6 profiles_to_run: ["burst", "energy", "intermittent", "mixed"]
7
8 failure_scenario:           # Used specifically for M_R
9   enable: true
10  time: 500.0                # seconds
11  affected_node: "LEO-1"
12  duration: 300.0            # seconds

```

Listing 2 – experiment.yaml

3.2 Shared State Schemas (src/schemas.py)

```

1 """Shared Pydantic state. Both teams code against these models."""
2 from enum import IntEnum
3 from pydantic import BaseModel, Field
4 from typing import Optional
5
6 class StrategyID(IntEnum):
7     ALWAYS_GROUND = 1
8     ALWAYS_ORBITAL = 2
9     GREEDY = 3
10    DYNAMIC = 4
11    RL_AGENT = 5
12
13 class TaskState(IntEnum):
14    PENDING = 0
15    SCHEDULED = 1
16    EXECUTING = 2
17    COMPLETED = 3
18    FAILED = 4
19    ORPHANED = 5 # Lost connection during execution
20
21 class Task(BaseModel):
22     id: str
23     arrival_time: float
24     cpu_units: float
25     ram_units: float
26     priority: int = 1 # 1=Low, 2=Urgent
27     data_size_mb: float = 1.0
28     state: TaskState = TaskState.PENDING
29     assigned_node: Optional[str] = None

```

```

30     start_time: Optional[float] = None
31     completion_time: Optional[float] = None
32
33 class NodeState(BaseModel):
34     id: str
35     cpu_capacity: float
36     cpu_utilized: float = 0.0
37     ram_capacity: float
38     ram_utilized: float = 0.0
39     energy_cost_per_cpu: float
40     active_tasks: list[Task] = Field(default_factory=list)
41     available: bool = True
42
43 class AllocationDecision(BaseModel):
44     task_id: str
45     chosen_node_id: str
46     score: Optional[float] = None
47     reason: str = ""
48
49 class RunTelemetry(BaseModel):
50     run_id: str
51     strategy: StrategyID
52     profile: str
53     metrics: dict # Contains M_T, M_L, M_R, M_E, M_S
54     decisions: list[AllocationDecision]
55     truncation_flag: bool = False

```

Listing 3 – schemas.py

3.3 Strategy Interface (src/strategies/base.py)

```

1 from abc import ABC, abstractmethod
2 from src.schemas import Task, NodeState, AllocationDecision
3
4 class AllocationStrategy(ABC):
5     """Every strategy implements exactly this interface."""
6
7     def __init__(self, nodes: dict[str, NodeState]):
8         self.nodes = nodes
9
10    @abstractmethod
11    def allocate(self, task: Task, current_time: float) ->
AllocationDecision:
12        """Given a task and current system state, decide where to
assign it."""
13        pass
14
15    @abstractmethod
16    def on_failure_detected(self, failed_node_id: str, orphaned_tasks:
list[Task], current_time: float) -> list[AllocationDecision]:
17        """Strategies 4 and 5 may re-allocate orphaned tasks."""
18        pass

```

Listing 4 – base.py

4 Strategy 5 and Visualization Extensions

4.1 Extending the Starter Code for Strategy 5

The RL agent will interact with the environment using the `stable-baselines3` interface.

```
1 from stable_baselines3 import PPO
2 from src.env.sim_env import SatelliteEnv # Gymnasium wrapper around
   SimPy
3 from src.strategies.base import AllocationStrategy
4
5 class RLAgentStrategy(AllocationStrategy):
6     def __init__(self, nodes, model_path=None):
7         super().__init__(nodes)
8         self.env = SatelliteEnv(nodes)
9         if model_path:
10             self.model = PPO.load(model_path, env=self.env)
11         else:
12             self.model = PPO("MlpPolicy", self.env, verbose=1)
13
14     def allocate(self, task, current_time):
15         observation = self.env.get_obs()
16         action, _states = self.model.predict(
17             observation,
18             deterministic=True,
19         )
20         return self.env.action_to_decision(action, task)
```

Listing 5 – `rl_wrapper.py` (Conceptual)

4.2 Interactive Dashboard

During Phase 3, `visualization/dashboard.py` will generate an interactive HTML dashboard using Plotly. It will show :

1. a live Gantt chart of task allocation across nodes;
2. the evolving Pareto Front as simulations progress;
3. radar charts comparing the five strategies on the four core metrics (M_T , M_L , M_R , and M_E), with M_S reported as a diagnostic stability metric.

This dashboard serves as the primary visual evidence for the final pitch.

4.3 Optional : Using Skyfield in Phase 3

For `benchmark_mode: "merged"`, adapt the following satellite-to-ground example in `src/env/link.py`. It tests elevation above the station's horizon, not the satellite's latitude alone. Supply station coordinates, a timezone-aware UTC simulation start, and a local TLE file containing one satellite. Use the same simulation start and archived TLE in every compared run, with dates close to the TLE epoch.

```
1 from datetime import timedelta, timezone
2 from skyfield.api import load, wgs84
3 from skyfield.iokit import parse_tle_file
4
5 class Link:
```

```

6     def __init__(
7         self, node1, node2, simulation_start,
8         station_latitude, station_longitude,
9         tle_file="data/tle.txt", min_elevation_deg=0.0,
10    ):
11        if simulation_start.utcoffset() is None:
12            raise ValueError("simulation_start must include a timezone")
13    )
14        self.node1 = node1
15        self.node2 = node2
16        self.start = simulation_start.astimezone(timezone.utc)
17        self.ts = load.timescale(builtin=True)
18        with open(tle_file, "rb") as tle_stream:
19            satellites = list(parse_tle_file(tle_stream, self.ts))
20        if len(satellites) != 1:
21            raise ValueError("Provide exactly one satellite in the TLE
22        file")
23        self.satellite = satellites[0]
24        self.station = wgs84.latlon(
25            station_latitude, station_longitude,
26        )
27        self.min_elevation_deg = min_elevation_deg
28
29    def is_active(self, current_time):
30        instant = self.start + timedelta(seconds=float(current_time))
31        skyfield_time = self.ts.from_datetime(instant)
32        relative_position = (self.satellite - self.station).at(
33            skyfield_time,
34        )
35        elevation, _, _ = relative_position.altaz()
36        return bool(elevation.degrees > self.min_elevation_deg)

```

Listing 6 – link.py (Skyfield integration)

Integration note : This is a geometric visibility check for a satellite-to-ground link, not a complete link model. Combine it with node availability and injected failures. Preserve the existing link behaviour for ground-to-cloud and inter-satellite links. The files listed under `data/` must be supplied separately; this template does not download them.

5 Minimal Working Example (Our First Task)

```

1 # notebooks/smoke_test.ipynb
2 from src.schemas import Task, NodeState, StrategyID
3 from src.strategies.s1_always_ground import AlwaysGroundStrategy
4 from src.env.topology import build_topology
5
6 # Build the fixed environment (from config)
7 nodes = build_topology()
8
9 # Instantiate Strategy 1
10 strategy = AlwaysGroundStrategy(nodes)
11
12 # Simulate a new task
13 task = Task(id="test-1", arrival_time=0.0, cpu_units=5.0, ram_units=10.0)
14 decision = strategy.allocate(task, current_time=0.0)

```

```
15
16 print(f"Task {task.id} assigned to {decision.chosen_node_id}. Reason: {
    decision.reason}")
17 # Expected output: Task test-1 assigned to Cloud-AWS. Reason: Always
    Ground.
```

Listing 7 – smoke_test.py

Week 1 goal for each team member : Make this smoke test pass for your assigned strategies using the mock topology, then integrate it with the SimPy `orchestrator.py` to simulate a timeline.